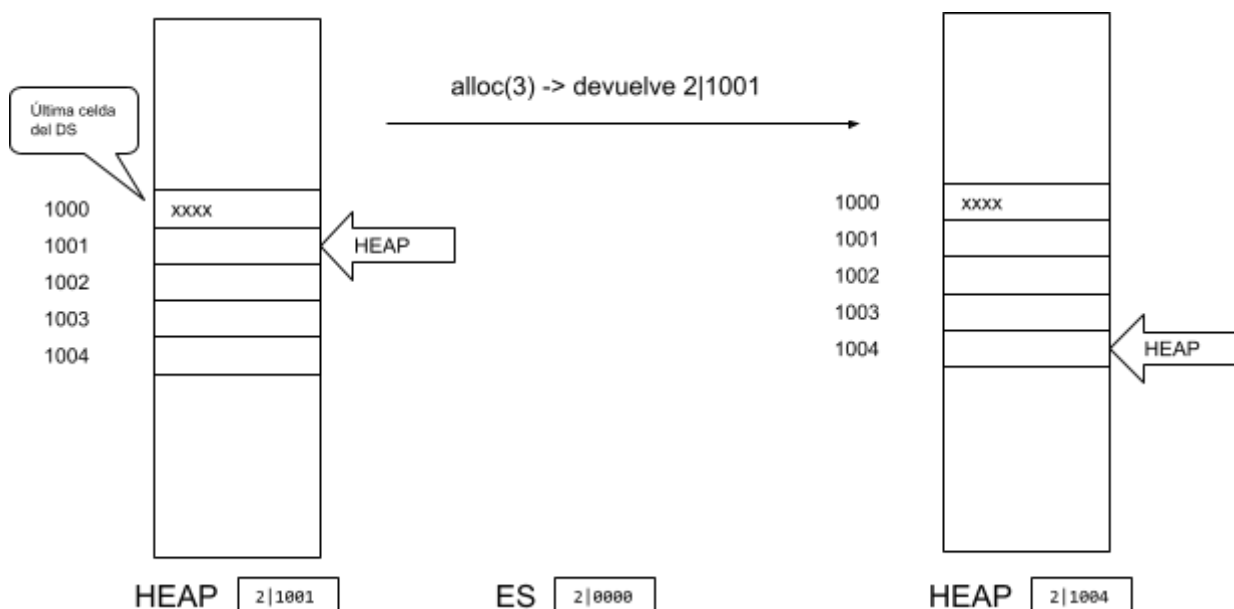


Guia de trabajos prácticos N° 6

Memoria dinámica y estructuras de datos

1. Crear una estructura "heap" para reservar memoria dinámicamente, que funciona de la siguiente manera:
 - a. Se utilizan los primeros 4 bytes del *Extra Segment (HEAP)* para almacenar el puntero (doble indirección) a la primera celda disponible del *Extra Segment* para memoria dinámica, que inicialmente será la siguiente.
 - b. Crear una subrutina **alloc(n)** donde n -pasado por parámetro- es la cantidad de bytes de memoria que se quiere reservar.
 - c. **alloc(n)** devuelve la dirección de memoria solicitada en el registro EAX e incrementa el contenido de la celda HEAP según "n" para apuntar al siguiente espacio de memoria disponible.



2. Utilizando alloc, crear subrutinas que permitan administrar listas dinámicas simplemente enlazadas de números enteros:
 - a. Crear nodo
 - b. Insertar nodo en forma ordenada
 - c. Remover nodo con valor pasado como parámetro
 - d. Buscar nodo
 - e. Imprimir la lista

La lista tiene la siguiente estructura:



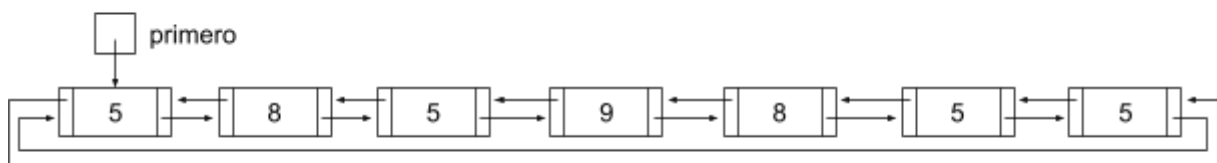
3. Traducir a assembler las siguientes funciones:

```
typedef struct _lista {
    short int dato;
    struct _lista* sig;
} lista;
```

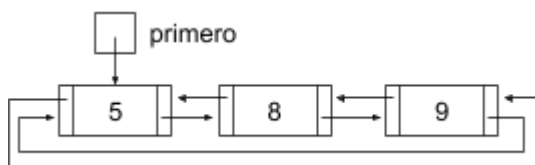
- a. `int ascendente(lista* L) {`
 `if (L == NULL) return 1;`
 `register lista* ant;`
 `do {`
 `ant = L;`
 `L = L->sig;`
 `} while (L != NULL && ant->dato <= L->dato);`
 `return L == NULL;`
`}`
- b. `lista** maximo(lista** L) {`
 `if (*L != NULL) {`
 `register list** max = maximo(&(*L)->sig);`
 `if (*max != NULL && (*L)->dato < (*max)->dato)`
 `L = max;`
 `}`
 `return L;`
`}`

4. Proponer nuevas implementaciones recursivas o iterativas, según corresponda, para las funciones del ejercicio anterior.
5. Escribir una subrutina que reciba dos listas de números (positivos, ordenados en forma creciente) y las intercale formando una lista ordenada.
- a. Creando una nueva lista.
- b. Reutilizando los mismos nodos (modificando las listas originales).
6. Crear una subrutina que invierta el orden de una lista simplemente enlazada.
7. Eliminar de una lista dinámica circular doblemente enlazada, todos los nodos con valor repetido. Además debe devolver la cantidad de nodos eliminados.

Lista inicial



Lista final



T_Lista
Y *1er nodo Puntero a celda tipo L,
ó -1 null (4 bytes)

Nodo Lista	
L	valor Entero (4 bytes)
L+4	*anterior Puntero a celda tipo L
L+8	*siguiente Puntero a celda tipo L

8. Crear una estructura de árbol binario que permita cargar ordenados números ingresados por el usuarios (indicando el último número con un número negativo). Armar subrutinas para:
- Agregar un nodo al árbol.
 - Mostrar el árbol inorden.
 - Mostrar el árbol preorden.
 - Mostrar el árbol postorden.
 - Solicitar un número e informar si el mismo se encuentra o no en el árbol.
 - Crear una subrutina que indique la altura que tiene el árbol.
9. Traducir a assembler las siguientes funciones:

```
typedef struct _arbol {
    short int dato;
    struct _arbol *izq, *der;
} arbol;
```

- a. `int suma_hijos(arbol* A) {`
 `register int res = 1;`

 `if (A != NULL && (A->izq != NULL || A->der != NULL)){`
 `short int suma = 0;`

 `if (A->izq != NULL)`
 `suma += A->izq->dato;`

 `if (A->der != NULL)`
 `suma += A->der->dato;`

 `res = A->dato == suma && suma_hijos(A->izq) && suma_hijos(A->der);`
 `}`
 `return res;`
}
- b. `arbol** maximo(arbol** A) {`
 `register arbol** max = A;`

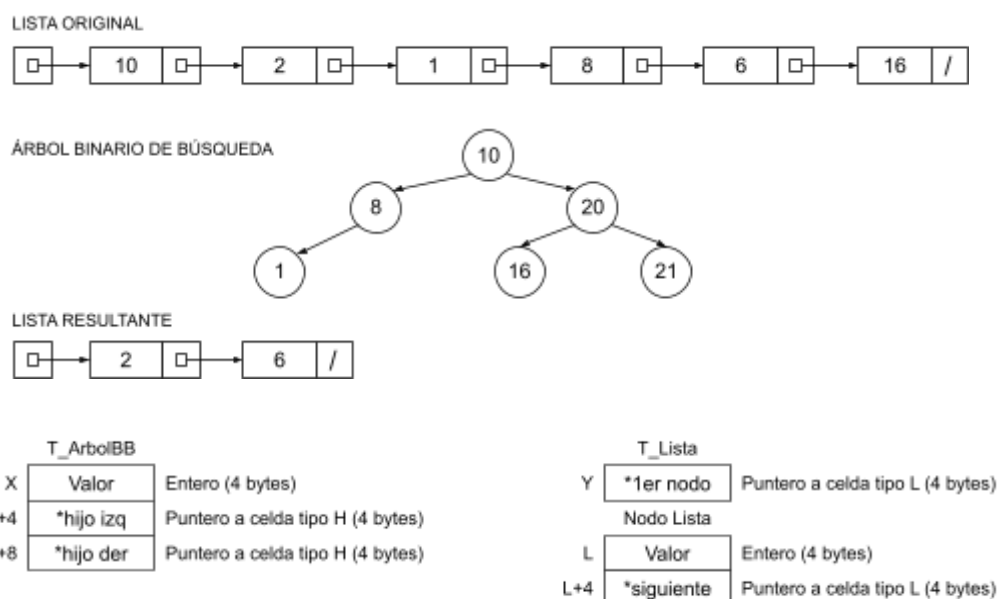
 `if (*A != NULL) {`
 `arbol** izq = maximo(&(*A)->izq);`
 `arbol** der = maximo(&(*A)->der);`

 `if (*izq != NULL && (*izq)->dato > (*max)->dato)`
 `max = izq;`

 `if (*der != NULL && (*der)->dato > (*max)->dato)`
 `max = der;`
 `}`

 `return max;`
}

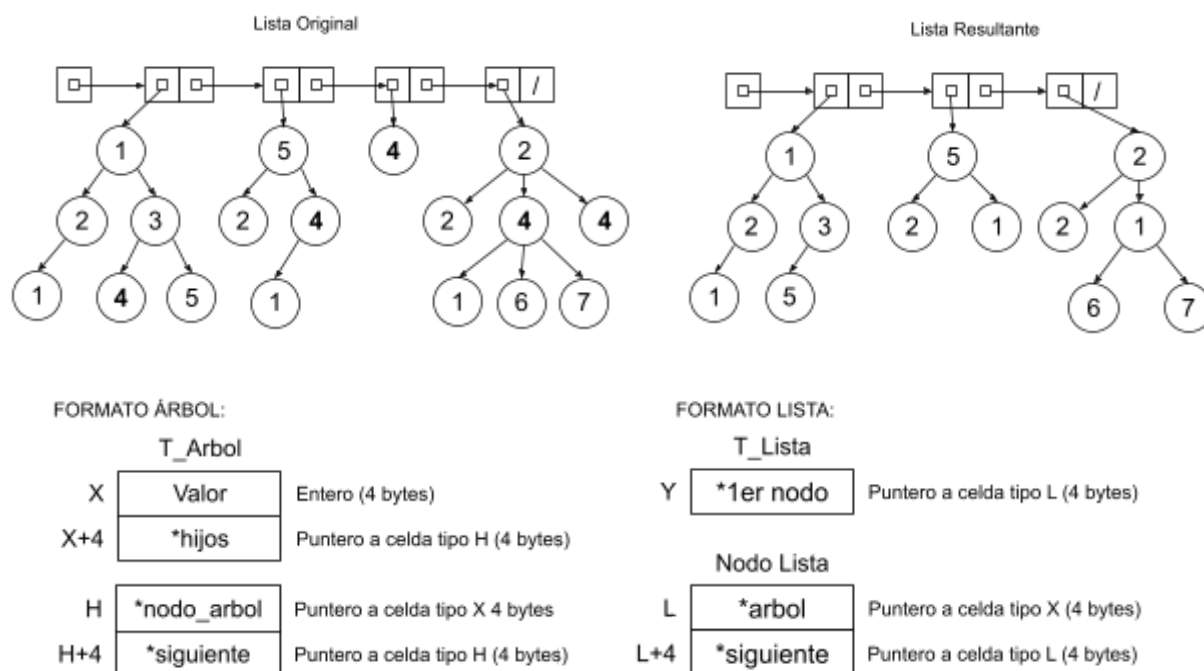
10. Dada una lista dinámica simplemente enlazada y un árbol binario de búsqueda, eliminar de la lista todos los nodos cuyo valor se encuentre en el árbol.



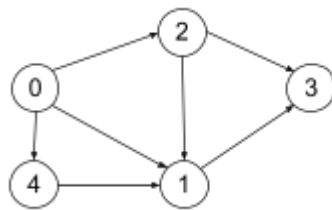
11. Dado un valor entero: eliminar de una lista dinámica simplemente enlazada, con punteros a árboles n-arios, todos los nodos de cada árbol con dicho valor. Cada nodo es reemplazado por su primer hijo. Además debe devolver la cantidad de nodos eliminados.

Debe tener en cuenta que si un nodo de la lista queda apuntando a un árbol vacío el mismo debe ser eliminado.

Ejemplo: valor de entrada 4, valor de salida 5, para la siguiente lista original:



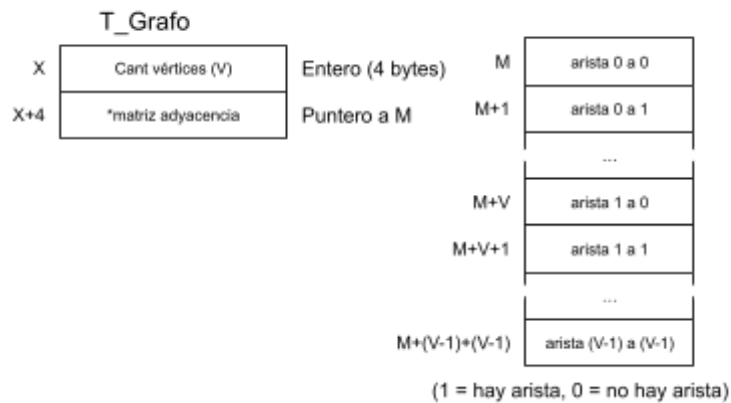
12. Contar la cantidad de caminos entre un vértice origen y uno destino de un digrafo acíclico.
Ejemplo:



origen = 0
destino = 3

Resultado:
*cantidad_caminos = 4

FORMATO GRAFO:



Representación del grafo del ejemplo:

5					
	0	1	2	3	4
0	0	1	1	0	1
1	0	0	0	1	0
2	0	1	0	1	0
3	0	0	0	0	0
4	0	1	0	0	0